



Academy of
Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

A Project Report on

Disaster Relief Volunteer Coordination System

Course	Enterprise Application Development (Advanced Java)
Problem Statement	Disaster Relief Volunteer Coordination / Real-time portal for task assignment, resource tracking, and SOS broadcast to nearest volunteers via socket programming.

Submitted by,

Name	PRN
Tushar Kaldate	202401040191
Abhay Ujade	202401040186

**A Report submitted to MIT Academy of Engineering, Alandi(D), Pune, An Autonomous Institute
Affiliated to Savitribai Phule Pune University**

Guided by,

Ms. Pushpmala V. Shinde

Department of Computer Engineering

MIT Academy of Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

Alandi (D), Pune – 412105

(2025–2026)

GitHub Link of Project : [tusharkkp/Java Mini Project](https://github.com/tusharkkp/Java_Mini_Project)

Contents

Sr. No.	Section Title	Page No.
1	Abstract	1
2	Problem Statement	2
3	Project Objectives	3
4	SDG Justification	4
5	Rubric Compliance Matrix	5
6	Technology Stack	6
7	System Features	7
7.1	Core Features	7
7.2	Technical Features	8
8	System Architecture	9
9	Project Architecture Overview	10
10	ER Diagram and Database Design	11
11	Class Diagram	12
12	Use Case Diagram	13
13	Implementation Highlights	14
14	Application Screenshots	15
15	SDG Metric Analysis	16
15.1	Metric Formulae	17
16	Testing and Expected Outcomes	18
17	Setup and Execution Instructions	19
18	Conclusion and Future Scope	20
19	References	21

Abstract

The Disaster Relief Volunteer Coordination System is a full-stack enterprise Java web application for coordinating emergency SOS requests, volunteer assignment, task tracking, and administrative monitoring. Users can create SOS requests with location and severity information. Volunteers receive alerts, manage availability, and update assigned task status. Administrators monitor all requests, assign available volunteers, and review SDG-oriented response metrics. The system follows a layered Spring MVC architecture with controllers, services, repositories, JPA entities, JSP/JSTL views, Spring Security roles, MySQL persistence, raw JDBC audit logging, asynchronous notifications, scheduled escalation checks, and Java socket programming for real-time SOS broadcasts. The project supports SDG 11 and SDG 13 by improving emergency response coordination and measuring response efficiency through average response time and percentage of requests handled within a target threshold.

1. Problem Statement

Design and implement a real-time disaster relief portal for task assignment, resource tracking, and SOS broadcast to nearest volunteers via socket programming. The system must support multiple secured roles, normalized database storage, MVC views, Hibernate/JPA persistence, Spring service and repository layers, JDBC fallback operations, multithreaded background processing, and measurable SDG impact reporting.

Project Objectives

- Allow affected users to raise SOS requests with severity, location, and description.
- Notify volunteers and administrators about new emergencies with asynchronous notifications and socket alerts.
- Enable administrators to assign volunteers and track the SOS lifecycle from NEW to ASSIGNED to RESOLVED.
- Provide SDG dashboard metrics such as average response time and percentage handled within threshold.
- Demonstrate enterprise Java concepts required by CO1 to CO4.

2. SDG Justification

The project aligns with SDG 11 because disaster response directly affects safe, resilient, and sustainable communities. Faster SOS intake, volunteer coordination, and response-time monitoring help cities reduce emergency impact on citizens. It also supports SDG 13 because climate-related floods, storms, heat events, and other disasters require coordinated emergency action. By broadcasting alerts, assigning volunteers, and measuring response performance, the application converts disaster response into a trackable digital workflow. The measurable SDG metric used in this project is response efficiency: average response time and percentage of resolved SOS requests handled within a 30-minute threshold.

3. Rubric Compliance Matrix

Rubric Area	Implementation Evidence
Requirement and Design	Problem statement, SDG linkage, ER diagram, class diagram, use cases, normalized schema, JPA relationships.
Back-end Development	@Controller, @Service, @Repository layers; Hibernate/JPA CRUD; HQL queries; @Transactional services; JDBC audit repository with PreparedStatement.
Front-end / Web Layer	JSP/JSTL views for login, dashboards, SOS forms, volunteers, users, tasks, and SDG metrics; Spring Security role access; RequestLoggingFilter.
Multithreading / Socket	@Async notification service; @Scheduled escalation/health checks; Java ServerSocket on port 9090 with multi-client handler threads.
SDG Integration	Admin SDG dashboard displays response threshold, average response time, resolved percentage, and volunteer availability.

4. Technology Stack

Layer	Technology
Presentation	JSP, JSTL, Spring MVC view resolver
Security	Spring Security with ROLE_ADMIN, ROLE_VOLUNTEER, ROLE_USER
Business Logic	Spring services, @Transactional, @Async, @Scheduled
Persistence	Hibernate/JPA entities, Spring Data JPA repositories, HQL queries
Database	MySQL with normalized tables and foreign keys
Networking	Java Socket/ServerSocket for live SOS alert broadcast
Audit	Raw JDBC PreparedStatement insert and batch operations

System Features :

Core Features:

- User Registration & Login using Spring Security.
- Role-based access control for ADMIN, VOLUNTEER, and USER.
- SOS Emergency Request creation with severity and location.
- Real-time SOS broadcast to volunteers using Java Socket Programming.
- Volunteer availability management.
- Task assignment and lifecycle tracking from NEW → ASSIGNED → COMPLETED.
- Distance-based volunteer selection using Haversine formula.
- Admin dashboard with operational monitoring and charts.

Technical Features:

- Asynchronous notification service using @Async.
- Scheduled escalation and health checks using @Scheduled.
- Raw JDBC operations with PreparedStatement for audit logging.
- HQL queries for distance-based filtering and SDG metrics.
- Servlet Filter for request logging.
- REST API endpoints for system interaction.
- SDG metrics dashboard for performance tracking.

5. System Architecture

The application follows a layered enterprise design. Controllers receive MVC or REST requests, services implement business workflows, repositories provide JPA/HQL/JDBC data access, and entities model the relational schema. The socket module runs separately from the HTTP port and broadcasts emergency alerts to connected Java clients.

Layer	Responsibilities	Main Classes
Controller	Routes requests and prepares models for JSP views.	AuthController, VolunteerController, RestApiController, SosController, AdminController,
Service	Business rules, notifications, assignments, metrics, scheduled jobs.	UserService, SosRequestService, TaskService, NotificationService, SdgMetricsService
Repository	Database CRUD, HQL queries, and JDBC audit operations.	UserRepository, SosRequestRepository, VolunteerRepository, JdbcSosLogRepository
Entity	JPA mappings, relationships, and lifecycle timestamps.	User, Role, Volunteer, SosRequest, Task, Notification, Location
Socket	Real-time Java TCP server and volunteer socket clients.	SosAlertSocketServer, VolunteerSocketClient

Project Architecture Overview :

The system follows a multi-layer architecture:

Controller Layer (@Controller, @RestController)



Service Layer (@Service, @Async, @Scheduled)



Repository Layer (@Repository, JpaRepository, JDBC)



Entity Layer (@Entity, JPA Annotations)



MySQL Database

This layered approach ensures separation of concerns, scalability, and maintainability of the application.

6. ER Diagram and Database Design

ER Diagram - Disaster Relief Volunteer Coordination System

Database entities, primary keys, foreign keys, and relationship cardinality

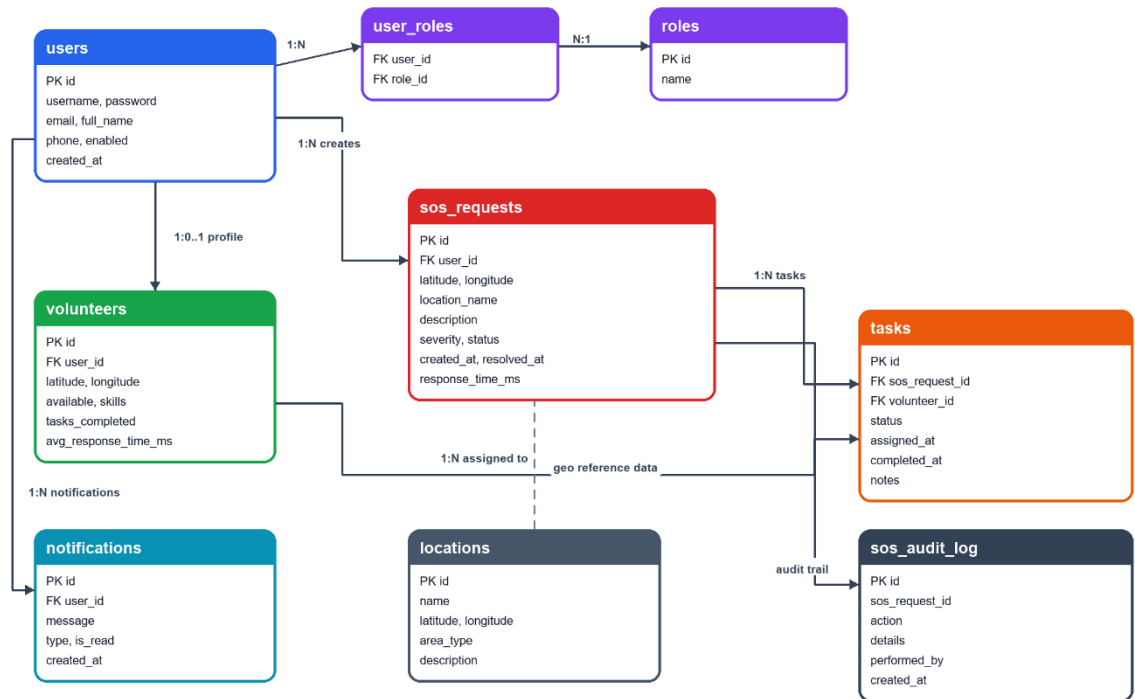


Figure 1. Normalized ER diagram showing user-role mapping, SOS requests, tasks, notifications, volunteers, locations, and audit logs.

Table	Purpose
users	Stores account identity, contact details, credentials, and enabled status.
roles / user_roles	Many-to-many role authorization mapping for Spring Security.
volunteers	Volunteer profile, location, availability, skills, and completed task count.
sos_requests	Emergency request data with location, severity, status, response time, and reporter.
tasks	Assignment records connecting SOS requests to volunteers with task lifecycle status.
notifications	User-facing alerts created asynchronously for SOS and task events.
locations	Predefined geographic areas for sample data and reporting.
sos_audit_log	Raw JDBC audit trail for create, assign, and status-change events.

7. Class Diagram

Class Diagram - Disaster Relief Application

Spring MVC layers, repositories, JPA entities, security, async processing, and socket integration

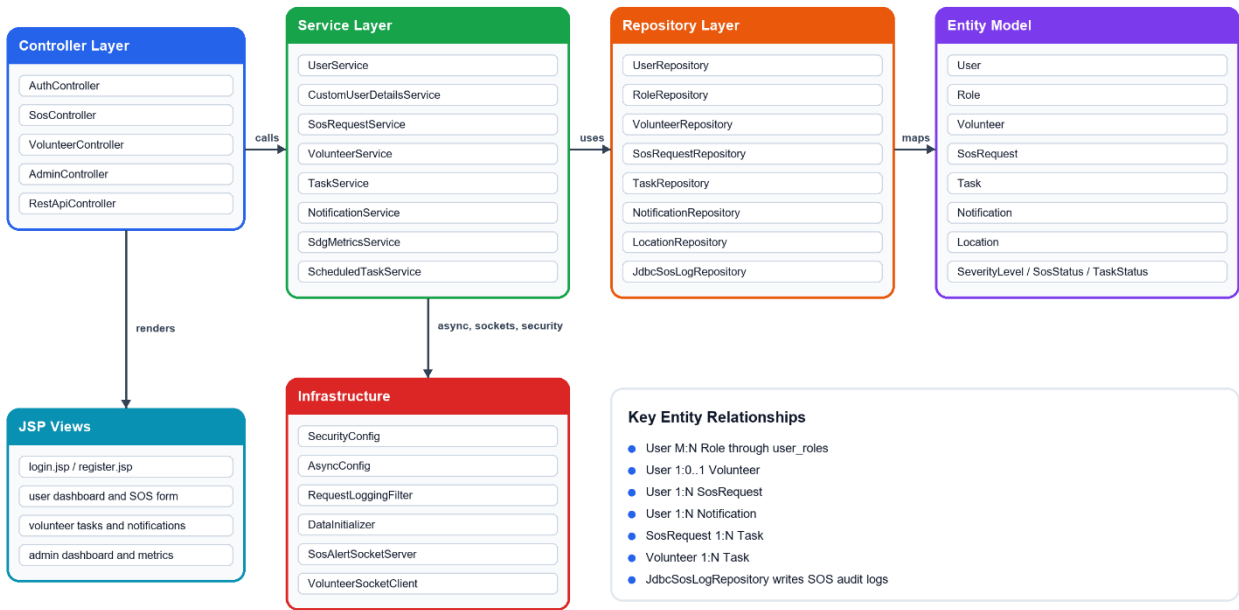


Figure 2. Layered class diagram highlighting controller, service, repository, entity, and infrastructure classes.

8. Use Cases

Use Case Diagram - Disaster Relief Coordination System

Role-based workflows for citizens, volunteers, and administrators

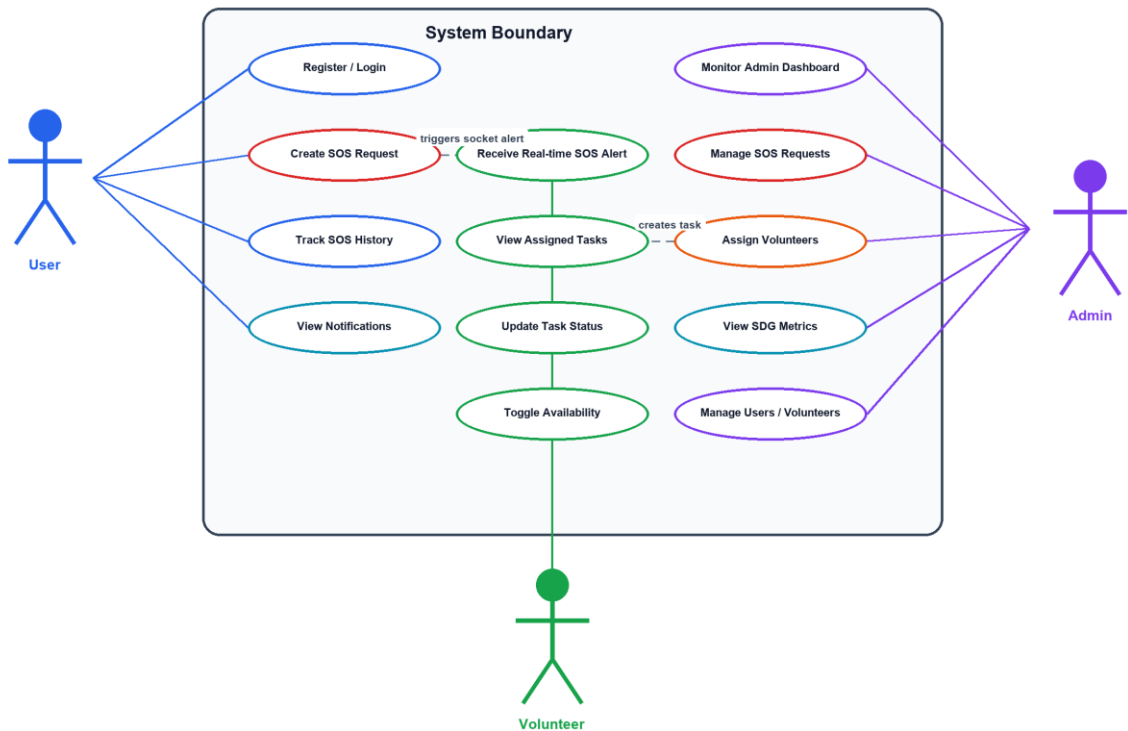


Figure 3. Role-based use case diagram for user, volunteer, and administrator workflows.

Actor	Use Case	Expected Outcome
User	Create SOS request	SOS is stored, volunteers are notified, socket alert is broadcast, and audit log is written.
Volunteer	View assigned tasks	Volunteer sees assigned tasks and updates status to IN_PROGRESS or COMPLETED.
Volunteer	Toggle availability and location	Availability and coordinates are updated for admin assignment decisions.
Admin	Assign volunteer	Task record is created and SOS status changes to ASSIGNED.
Admin	Monitor SDG metrics	Dashboard shows average response time and percentage handled within threshold.

9. Implementation Highlights

Spring Security Role-Based Routing

```
.requestMatchers("/admin/**").hasRole("ADMIN")  
.requestMatchers("/volunteer/**").hasRole("VOLUNTEER")  
.requestMatchers("/user/**").hasRole("USER")
```

The security filter chain enforces separate dashboards and workflows for administrators, volunteers, and regular users.

JPA Entity Relationship

```
@ManyToMany(fetch = FetchType.EAGER)  
@JoinTable(name = "user_roles", joinColumns =  
@JoinColumn(name = "user_id"),  
inverseJoinColumns = @JoinColumn(name =  
"role_id")) private Set<Role> roles = new  
HashSet<>();
```

The user-role mapping supports many-to-many authorization and demonstrates Hibernate relationship annotations.

Raw JDBC Audit Logging

```
String sql = "INSERT INTO sos_audit_log " +  
"(sos_request_id, action, details, performed_by, created_at) " +  
"VALUES (?, ?, ?, ?, ?)";  
PreparedStatement pstmt = conn.prepareStatement(sql);
```

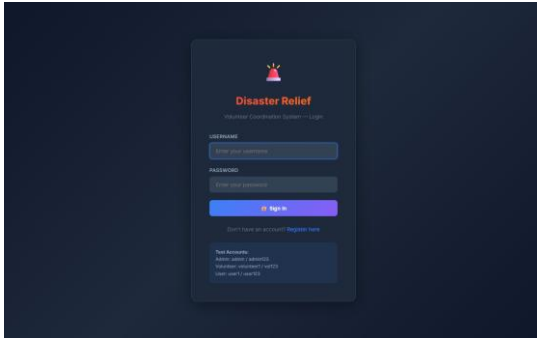
The audit repository satisfies the JDBC requirement by using PreparedStatement parameter binding outside JPA.

Multithreading and Socket Alert

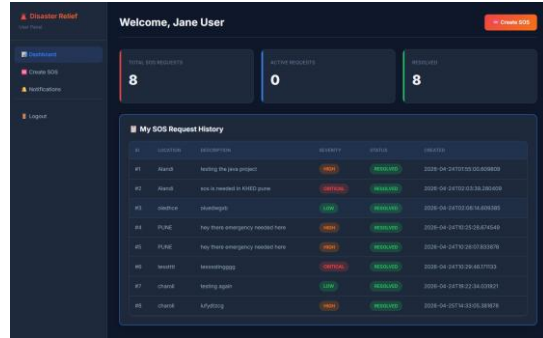
```
@Async public void broadcastToRole(String roleName, String message, String type)  
{ ... }  
  
new Thread(new ClientHandler(clientSocket)).start();  
socketServer.broadcastMessage(socketMessage);
```

Notifications are sent asynchronously and each socket client is handled on a separate thread, supporting live SOS alerts.

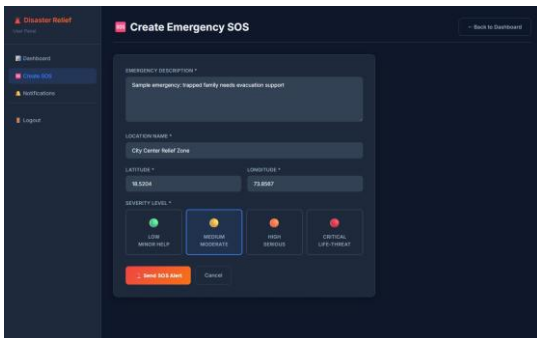
10. Application Screenshots



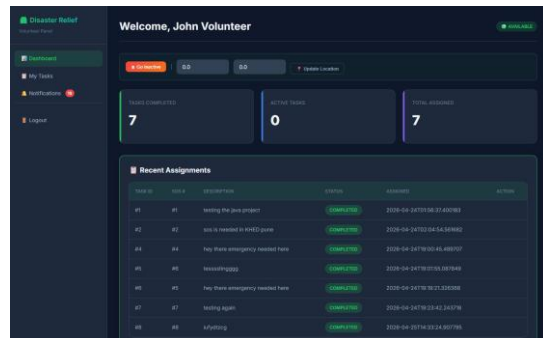
Login page with default test accounts.



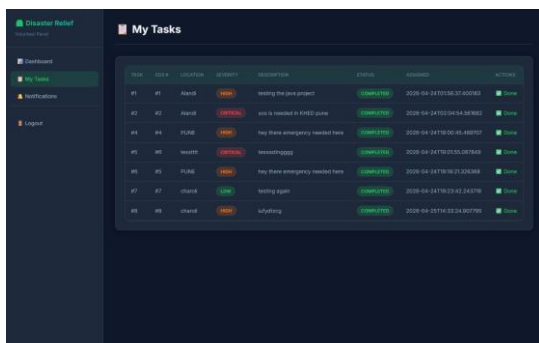
User dashboard showing SOS history and quick actions.



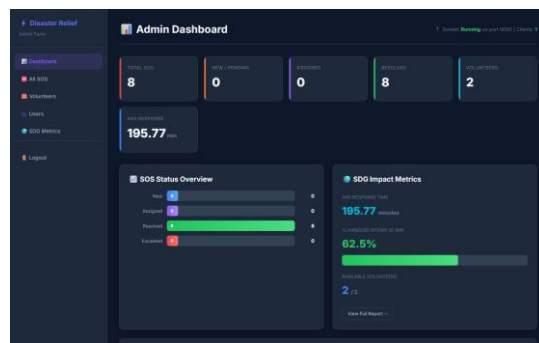
Create SOS form with location, description, and severity



Volunteer dashboard with availability and assigned work selection. summary.



Volunteer task management screen.



Admin dashboard showing operational and socket metrics.

ID	User	Location	Status	Description	Priority	Actions
P1	John Doe	London	Pending	Testing the app project	High	Assign
P2	John Doe	London	In Progress	App is installed on RPI32 device	Medium	Assign
P3	John Doe	London	Pending	App is installed on RPI32 device	Medium	Assign
P4	John Doe	London	Pending	App is installed on RPI32 device	Medium	Assign
P5	John Doe	London	Pending	App is installed on RPI32 device	Medium	Assign
P6	John Doe	London	Pending	App is installed on RPI32 device	Medium	Assign
P7	John Doe	London	Pending	App is installed on RPI32 device	Medium	Assign
P8	John Doe	London	Pending	App is installed on RPI32 device	Medium	Assign

Admin SOS list for reviewing and assigning requests.



SDG impact dashboard with response performance metrics.

11. SDG Metric Analysis

The SDG dashboard converts disaster response activity into measurable indicators. The current data snapshot was read from the running REST endpoint `/api/v1/metrics` and is shown below.

Metric	Current Value	Interpretation
Total SOS Requests	8	Total recorded emergencies handled by the system.
Resolved SOS Requests	8	Requests moved to resolved state after volunteer task completion.
Average Response Time	195.77 minutes	Mean time from SOS creation to first assignment/resolution tracking point.
Within Threshold	62.5%	Resolved SOS requests handled within 30 minutes.
Available Volunteers	2/2	Active volunteer capacity visible to administrators.
Completed Tasks	8	Task closure count, supporting operational accountability.

Metric Formulae

- Average response time = $\text{average}(\text{response_time_ms}) / 60000$.
- Within-threshold percentage = $\text{resolved SOS within threshold} / \text{total resolved SOS} * 100$.
- Volunteer availability = $\text{available volunteers} / \text{total volunteer profiles}$.

The metric analysis indicates whether the relief workflow is improving response speed and volunteer utilization. A lower average response time and a higher within-threshold percentage indicate better SDG-aligned resilience.

12. Testing and Expected Outcomes

Test Area	Expected Result
Build	<code>.\mvnw.cmd</code> clean compile completes with BUILD SUCCESS.
Context Test	<code>.\mvnw.cmd</code> test loads Spring context with MySQL and passes.
Login	Admin, volunteer, and user accounts redirect to their role dashboards.
SOS Creation	User creates SOS; request is saved, notifications are created, socket broadcast occurs.
Assignment	Admin assigns volunteer; task is created and SOS status changes.
Volunteer Task	Volunteer updates task status; completed tasks resolve the SOS when all linked tasks are complete.
Socket	Socket server runs on port 9090 and client count changes when Java socket clients connect/disconnect.
REST API	<code>/api/v1/sos</code> , <code>/api/v1/volunteers</code> , <code>/api/v1/tasks</code> , <code>/api/v1/metrics</code> return JSON.

Setup and Execution Instructions

Prerequisites:

- Java 21 or higher
- MySQL 8.x

Database Setup:

```
CREATE DATABASE disaster_relief_db;
```

Run Application:

Windows:

```
.\mvnw.cmd spring-boot:run
```

Linux/Mac:

```
./mvnw spring-boot:run
```

Access:

- Web Application: <http://localhost:8081>
- Socket Server: Port 9090

13. Conclusion and Future Scope

The project demonstrates an enterprise Java solution for disaster relief coordination using Spring MVC, Hibernate/JPA, JSP/JSTL, Spring Security, MySQL, JDBC, multithreading, and Java socket programming. It satisfies the course outcomes by combining web development, persistence, concurrency, and real-time networking in a socially relevant SDG context.

- Add map-based distance visualization and route estimation.
- Replace raw TCP demo clients with browser WebSocket support for production usability.
- Add mobile-friendly volunteer check-in and push notifications.
- Add richer analytics such as severity-wise response time and volunteer workload balance.
- Deploy with cloud database backups and production CSRF/session-hardening settings.

14. References

- Enterprise Application Development Mini Project Problem Statements and Assessment Rubric, Academic Year 2025-26.
- Spring Boot Reference Documentation: <https://docs.spring.io/spring-boot/>
- Spring Framework MVC Documentation: <https://docs.spring.io/spring-framework/reference/web/webmvc.html>
- Spring Security Reference: <https://docs.spring.io/spring-security/reference/>
- Hibernate ORM User Guide: <https://docs.jboss.org/hibernate/orm/>
- Jakarta Tags / JSTL Specification: <https://jakarta.ee/specifications/tags/>
- MySQL Documentation: <https://dev.mysql.com/doc/>
- Java SE Networking API: <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/net/package-summary.html>
- United Nations SDG 11: <https://sdgs.un.org/goals/goal11>
- United Nations SDG 13: <https://sdgs.un.org/goals/goal13>